

Initial Network (Step 6)

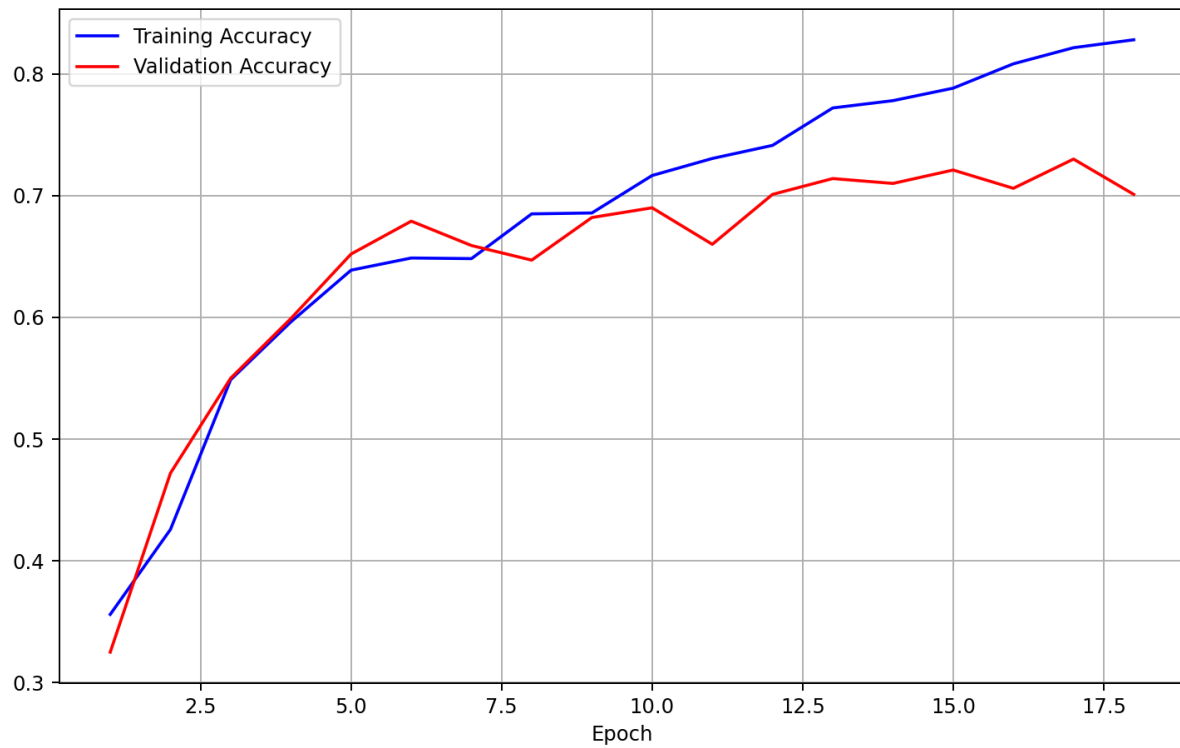
Model: "sequential"

Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 150, 150, 3)	0
conv2d (Conv2D)	(None, 148, 148, 16)	448
max_pooling2d (MaxPooling2D)	(None, 74, 74, 16)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 32)	0
dropout (Dropout)	(None, 36, 36, 32)	0
conv2d_2 (Conv2D)	(None, 34, 34, 32)	9248
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 32)	0
flatten (Flatten)	(None, 9248)	0
dense (Dense)	(None, 3)	27747
Total params: 42,083		
Trainable params: 42,083		
Non-trainable params: 0		

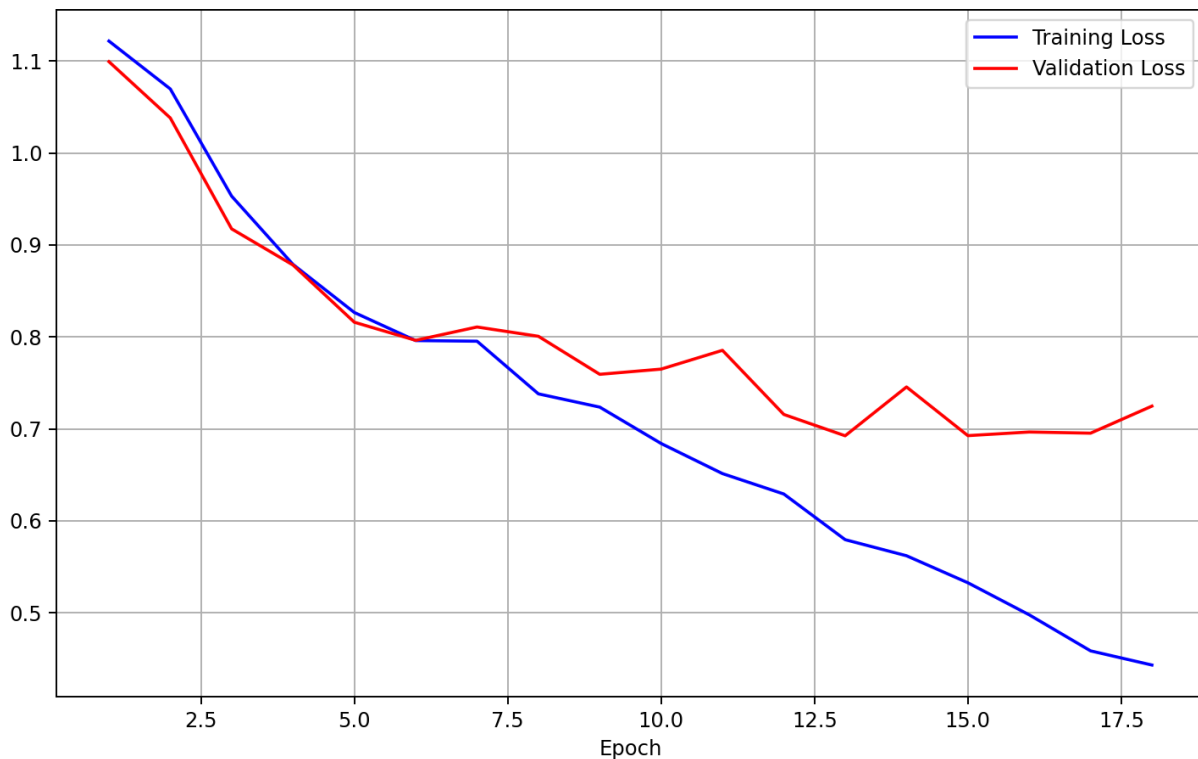
* Evaluating basic_model

30/30 [=====] - 2s 55ms/step - loss: 0.7013 - accuracy: 0.7113

Training and Validation Accuracy / Epoch (Step 6)



Training and Validation Loss / Epoch (Step 6)



Hyper Parameter Optimization Strategy

The approach we took to optimize the model was primarily adjusting the convolutional layers. We also tested a few different approaches to using the dropout layers which involved testing a couple different drop rates and where they were positioned in the network. The most notable effect that we observed was the way in which the validation accuracy dips depending on the rate assigned to the dropout layer, which if assigned too high, creates drastic shifts in the validation accuracy. This makes sense since throwing out and generalizing over large amounts of data could create big shifts in perceived accuracy.